



NO CAP

Product Requirements Document

Design it. Break it. Scale it. — for one learner, at zero cost.

DOCUMENT	Product Requirements Document
VERSION	v2.1 (personal app, zero-cost)
DATE	16 August 2026
SCOPE	Single-user learning workbench
DEPLOYMENT	Cloudflare-native, zero-cost (no paid deps, no accidental billing)
STATUS	Working spec

PRODUCT THESIS

A personal system-design workbench that turns passive reading into a repeatable loop of learning, visualization, reasoning, simulation, architecture practice, and interview readiness — with no paid dependencies and no accidental billing, on Cloudflare's genuine free tier, where core functionality remains usable at free-tier exhaustion.



What Changed: v2.0 to v2.1

v2.0 was an excellent product vision but assumed a production deployment with managed PostgreSQL, Redis, Celery workers, paid TTS/STT providers, live cloud pricing APIs, and remote code execution. v2.1 rewrites the architecture for a single requirement: **zero monthly cost, forever, with no trial credits and no accidental billing**. Every feature from v2.0 is preserved as a vision, but features are now classified into four tiers and sequenced so the core learning loop ships first on infrastructure that costs nothing.

Architectural Pivots

- PostgreSQL → Cloudflare D1 (SQLite, 5GB free).
- Redis → removed. Caching via KV; presence via Durable Objects.
- Celery/RQ → Cloudflare Queues + Cron Triggers.
- S3 → Cloudflare R2 (10GB free, free egress).
- FastAPI server → FastAPI on Python Workers (Pyodide).
- WebSocket + Yjs → Durable Objects + WebSockets (free tier).
- Managed TTS/STT → browser-native first, external optional.
- Live cloud pricing APIs → versioned pricing dataset bundled.
- Remote code execution → deferred. Code walkthrough mode replaces it.
- AI mentor → optional, Workers AI with quota guards.

Scope Discipline

- Product identity locked: "the system-design gym", not LinkedIn+Notion+LeetCode.
- Every feature classified Tier A / B / C / D (see § 6).
- Sequencing: 0.1 Learning Engine → 0.5 Design Gym → 1.0 Workbench → 2.0 Multiplayer.
- Web/PWA is primary. Mobile is read/review-only initially.
- Content is mostly static — versioned JSON/MDX in git, not a CMS.
- Zero-cost is a non-functional requirement, not an aspiration.
- Quota guards fail-closed: feature unavailable, not billed.
- Single-user auth — no multi-tenant complexity in MVP.

THE HONEST TRUTH ABOUT FREE TIERS

Cloudflare's free tier is genuine and explicitly "fail-closed" (queries fail rather than billing you), but it is not infinite. 100k Worker requests/day, 5M D1 rows read/day, 10k Workers AI neurons/day — these are real numbers. The correct architecture is "free by construction, never accidentally billable", not "pretend free tiers are infinite". Every optional service has a quota guard, a fallback, and a graceful-degradation path baked in from day one.

1. Product Overview

NO CAP is a personal system-design learning workbench. It combines a structured roadmap, daily learning, first-party lessons, visual explanations, interactive simulations, quizzes, an architecture playground, case studies, spaced review, projects, and interview practice in one coherent experience — optimized for a single learner (the author) and deployable for zero monthly cost on Cloudflare's genuine free tier.

The core content, diagrams, flowcharts, simulations, quizzes, and progression logic are owned by the product and stored as versioned JSON/MDX in a git repository. User-specific state (progress, mastery, notes, attempts, saved architectures) lives in Cloudflare D1. Media assets live in R2. Realtime collaboration (a Tier B feature) uses Durable Objects. Optional services (AI, TTS, STT) are feature-gated behind quota guards and degrade gracefully to deterministic fallbacks when their free quotas are exhausted.

2. Product Vision

Make system design something the learner repeatedly does, not just something they read about. Move from definitions to mental models, from mental models to architecture decisions, and from architecture decisions to confident interviews and real shipped systems — all within a personal workbench that costs nothing to run.

NORTH STAR

A learner should be able to explain a concept, visualize it, choose when to use it, reason about failure and trade-offs, estimate what it costs to run, and apply it in a system design interview — all within the same learning loop, on infrastructure that bills zero dollars per month.

PRODUCT IDENTITY LOCK

NO CAP is **the system-design gym**. Not LinkedIn. Not Notion. Not LeetCode. Not Figma. Not Coursera. Not an AI app. Every feature must reinforce the loop: learn → visualize → reason → design → break → review. If a feature does not strengthen that loop, it is pushed down the roadmap regardless of how cool it sounds.

3. Target User

v2.0 listed eight personas. v2.1 collapses to one: **the author**. This is a personal app, not a product. There is exactly one user. There is no onboarding flow, no roles system, no admin panel, no sharing UX to design, no rate-limiting per-user, no multi-tenancy. The schema can assume `user_id = 1` forever (or omit `user_id` entirely from MVP tables and add it later if the app ever goes multi-user).

This single change eliminates enormous surface area: auth becomes "log in with GitHub OAuth for myself only", admin becomes "edit the JSON files in the repo", analytics becomes "I look at the dashboard", and most importantly, quota guards only need to protect one user's

usage against Cloudflare's daily limits — which is a much easier problem than protecting thousands of users.

4. Product Principles

Content-first and content-static. First-party core learning lives as versioned JSON/MDX in git, built into static bundles at deploy time. User state is the only dynamic data. This keeps the database small and the Worker requests cheap.

Depth over volume. Each core concept explains problem, mechanism, trade-offs, failure modes, real use, and (where relevant) the code that implements it. Never ship a concept without all seven definition-of-done items.

Visual by default. Diagrams, flows, timelines, state transitions, and interactive models whenever they improve comprehension. Static text is the fallback, not the default.

Application over completion. Progress is based on demonstrated understanding, not page views. The product rewards building, breaking, and explaining — never passive scrolling.

Trade-offs are first-class. The platform repeatedly asks why, why not, what breaks, and what it costs. Every architecture decision is paired with a cost estimate (from bundled pricing data) and a failure mode.

One concept, many representations. Lesson, visual, simulation, quiz, scenario, architecture use, code walkthrough, interview question — all feeding the same mastery signal.

Low-noise AI. AI is an optional tutor and reviewer, not a substitute for authored curriculum. AI output never silently becomes official course content. When Workers AI quota is exhausted, the app shows authored fallbacks — never a paywall.

Free by construction. Every external dependency has a deterministic fallback. Quota guards fail-closed. No service can ever produce a bill. This is a non-functional requirement, not an aspiration.

Modern, non-decorative UI. Glassmorphism and depth serve comprehension; they are not chrome. Color carries meaning (warm amber for focus, forest green for mastery, rust for trade-off caution). The product never looks like a generic AI template.

5. Learning Model

The learning loop is intentionally cyclical. Every concept passes through seven states, and a concept can regress into Review Due if recall, application, or explanation performance drops over time. The model is unchanged from v2.0 — it was already right.

DISCOVER → UNDERSTAND → VISUALIZE → PRACTICE → APPLY → RECALL → REASSESS

Mastery states: Not Started, Exposed, Understood, Practiced, Applied, Review Due, Mastered. v2.1 defers the "Annotated" and "Cost-Aware" sub-states from v2.0 to Tier B (they require Notes and Cost Calculator features respectively). A concept is Mastered when the learner has demonstrated recall, application, and explanation at configured thresholds.

6. Feature Tier Classification

Every feature is classified into one of four tiers. Tier A is the must-have core. Tier B is strong V2. Tier C is optional integration (never an architectural dependency). Tier D is deferred. The full v2.0 vision is preserved — but the build sequence respects the tiers.

Tier	Meaning	Build When	Cost Constraint
A - Core	Must-have. The product does not exist without these.	First. Versions 0.1 and 0.5.	Zero-cost infrastructure only. No external dependencies.
B - Strong V2	High-value features that round out the workbench.	After A is solid. Version 1.0.	Zero-cost infrastructure. Optional services allowed if quota-guarded.
C - Optional Integration	Nice-to-have. App works perfectly without them.	Whenever. Never blocks core.	Optional services with hard quota guards + deterministic fallback.
D - Deferred	Vision-aligned but not now. Revisit when constraints change.	Future. Not in any current version.	Re-evaluate infrastructure cost at that time.

7. Core Product Modules by Tier

Every module is tagged with its tier and target version. The Status column shows where it lands in the 0.1 / 0.5 / 1.0 / 2.0 sequence.

Module	Purpose	Tier	Version
Home / Today	Daily dose, review queue, recommended next concept, streak	A	0.1
Roadmap	Structured curriculum across phases and concept dependencies	A	0.1

Module	Purpose	Tier	Version
Concept Library	Searchable encyclopedia of system-design concepts	A	0.1
Concept Lesson	Deep content with diagrams, examples, trade-offs, misconceptions	A	0.1
Quiz Engine	MCQ, multi-select, ordering, prediction, scenario questions	A	0.1
Review / Spaced Repetition	Recall cards, concept checks, scheduled reviews	A	0.1
Progress / Mastery	Skill matrix, weak areas, concept graph, milestones	A	0.1
Glossary	Fast definitions and cross-links	A	0.1
Focus Mode	Pomodoro timer, ambient sound, distraction-free surface	A	0.1
Interactive Labs	Change variables and see system behavior (client-side)	A	0.5
Architecture Playground	Drag, connect, annotate, validate architecture	A	0.5
Case Studies	Guided real-world system designs and reference architectures	A	0.5
Interview Mode (text)	Timed system-design sessions with progressive constraints	A	0.5
Notes & Annotations	Personal notes, highlights, knowledge graph	B	1.0
Collections & Study Lists	Custom concept packs with system starters	B	1.0
Cost Calculator	Estimate cloud spend using bundled pricing dataset	B	1.0
Career Path Mapping	Map concepts to roles, gap analysis, plan generator	B	1.0
Code Walkthrough Mode	Read, modify, predict — no execution (replaces sandbox)	B	1.0
Export & Share	PNG/SVG/Mermaid/Excalidraw export, shareable URLs	B	1.0
Real-World Teardowns	Engineering blog teardowns, postmortem walkthroughs	B	1.0
Projects	Hands-on build challenges mapped to concepts	B	1.0

Module	Purpose	Tier	Version
Collaborative Whiteboarding	Multiplayer architecture design via Durable Objects	B	2.0
Voice / Audio Mode	Browser TTS for lessons, hands-free review (Web Speech API)	C	2.0
Voice Interviewer	Browser STT + TTS for spoken interview practice	C	2.0
AI Mentor	Optional tutor/reviewer via Workers AI (quota-guarded)	C	2.0
Calendar Sync	Google/Microsoft calendar integration for session scheduling	C	2.0
External Resource Previews	Cached link previews via Worker fetch with timeout	C	2.0
Code Sandbox (live execution)	Remote code execution via Firecracker/e2b	D	Future
Live Cloud Pricing APIs	Real-time AWS/GCP/Azure pricing refresh	D	Future
Mobile App (native)	React Native + Expo native shell	D	Future
Multi-user / Cohorts	Multi-tenant transformation, sharing, leaderboards	D	Future

8. Curriculum Scope

The initial curriculum is anchored to the user-supplied roadmap.sh system-design roadmap and expanded with the missing foundations. v2.1 keeps the v2.0 scope but adds a note: the Sustainability and Platform Engineering additions from v2.0 are Tier B content — they ship after the core 14 areas are solid.

Area	Coverage	Tier
Fundamentals	How the internet works, DNS, request-response, client-server, application/data layer, requirements, scaling from one user to many.	A
Scaling	Vertical vs horizontal scaling, bottlenecks, load balancing, statelessness, replication, sharding, service boundaries, CAP in practice.	A
Networking	HTTP, TCP, UDP, TLS, HTTP/2, HTTP/3, QUIC, WebSockets, SSE, proxies, reverse proxies, NAT.	A
Databases	SQL vs NoSQL, key-value, document, wide-column, graph, indexes, transactions, isolation, MVCC, WAL, replication, sharding, denormalization, tuning, OLTP/OLAP.	A
Caching	Strategies, cache-aside, write-through, write-behind, refresh-ahead, client/CDN/web/database/application caching, invalidation and stampede control.	A
Asynchronism	Queues, pub/sub, event-driven architecture, competing consumers, fan-out, backpressure, task queues, async request-reply.	A
Distributed Systems	Consistency models, quorum, leader election, consensus, logical clocks, partitions, retries, idempotency, exactly-once myths, distributed transactions.	A
Reliability	Health checks, timeouts, retry, circuit breaker, bulkhead, throttling, graceful degradation, failover, disaster recovery, multi-region thinking.	A
Observability	Logs, metrics, traces, SLIs, SLOs, SLAs, golden signals, instrumentation, alerting, incident analysis.	A
Security	Authentication, authorization, OAuth/OIDC, IAM, RBAC/ABAC, TLS, secrets, WAF, DDoS, zero trust, network isolation.	A
Cloud Design	Compute, managed databases, object storage, CDN, load balancers, API gateways, VPC concepts, cost-aware architecture; AWS examples as a dedicated cloud track.	A
Real-time Systems	TCP vs UDP, WebSockets, WebRTC, SFU/MCU/P2P, live streaming, RTMP, media pipelines.	A

Area	Coverage	Tier
AI Systems	RAG, vector stores, context management, workflow engines, AI application observability, real-time voice AI architecture, fine-tuning ops, evals, drift detection, guardrails.	A
Design Patterns	CQRS, event sourcing, sidecar, strangler fig, gateway patterns, anti-corruption layer, leader election, pipes and filters, external config.	A
Interview & Case Studies	URL shortener, Instagram, WhatsApp, Uber, Netflix, YouTube, ride sharing, hotel reservation, rate limiter, key-value store, consistent hashing.	A
Sustainability	Carbon-aware computing, green software patterns, scheduling workloads around carbon intensity, embodied vs operational carbon, sustainability KPIs.	B
Platform Engineering	Internal Developer Platforms, golden paths, Backstage-style portals, scaffolding, environment as a service, developer productivity metrics.	B

9. Daily Dose Requirements

The Daily Dose is the heart of habit formation. v2.1 keeps the v2.0 spec but removes the voice and cost micro-question options from MVP — they reappear as Tier C and Tier B features respectively in later versions. MVP dose is strictly text + visual + interactive + quiz + recall.

- Default session length: 10-15 minutes; user-selectable Lite (5 min), Standard (12 min), Deep (25 min).
- A daily session contains: one new concept, one review concept, one quiz, one visual, one mini scenario.
- The recommendation engine avoids introducing a concept before its key prerequisites unless the user explicitly overrides it.
- Every completed dose records learning events that feed mastery and spaced repetition.
- Users can snooze or skip without breaking the roadmap; the system recovers gracefully rather than punishing missed days.
- Focus Mode can be toggled on for any Daily Dose: full-screen surface, ambient sound option, Pomodoro timer visible, no notifications. (Tier A)
- Optional Voice Mode (Tier C, v2.0): the entire dose is playable as audio via browser TTS. Deferred to v2.0 release.
- Optional Cost micro-question (Tier B, v1.0): a single quick cost reasoning prompt. Deferred to v1.0 release.

10. Interactive Learning Requirements

Labs are deterministic simulations executed entirely client-side. v2.1 keeps the v2.0 lab families but moves the new "Cost Simulator" output to use bundled pricing data (not live APIs). The "Failure Injector" and "Capacity Planner" labs remain Tier A — they are pure client-side computation, costing nothing to run.

Lab Type	Example Variables	Observed Output	Tier
Load balancing	Users, request rate, server count, algorithm, node failure	Distribution, queueing, failures, p95/p99 latency	A
Caching	TTL, cache size, hit rate, eviction policy	Hit ratio, backend load, freshness	A
Replication	Replica count, read/write ratio, lag	Read availability, stale reads, failover behavior	A
Sharding	Shard count, key distribution, hot key	Balance, hotspots, query routing	A

Lab Type	Example Variables	Observed Output	Tier
Queues	Producer rate, consumer rate, workers, message size	Backlog, throughput, processing delay	A
Rate limiting	Limit, burst, algorithm, clients	Allowed/rejected requests, fairness	A
CDN	Cacheability, TTL, origin rate, geography	Origin load, edge hit rate, latency	A
Failure Injector	Component to kill, duration, blast radius	User-visible impact, cascading failures, recovery time	A
Capacity Planner	Growth rate, seasonality, headroom target	When to scale, predicted bottleneck, recommended actions	A
Cost Simulator	Service mix, region, reserved vs on-demand, traffic pattern	Monthly \$ estimate (from bundled pricing), cost breakdown, cost-per-request	B

11. Architecture Playground Requirements

The Architecture Playground is the core practice surface. v2.1 keeps the v2.0 spec but resequences: cost annotation is Tier B (v1.0), collaborative editing is Tier B (v2.0), version history and export are Tier B (v1.0). The Tier A MVP is solo editing + deterministic validation + save as JSON graph in D1.

- [A] Component palette: client, DNS, CDN, load balancer, gateway, service, cache, DB, replica, queue, worker, object storage, search, auth, monitoring.
- [A] Edges must represent request, event, dependency, or data flow; edge type is visible.
- [A] Nodes have configurable properties relevant to the exercise (instance type, replica count, region).
- [A] The validator detects common issues: single points of failure, unnecessary synchronous chains, missing caching, bad data ownership, unsafe public exposure, overloaded components, and obvious scaling mismatches. Runs entirely client-side.
- [A] The system provides hints in learning mode and stricter evaluation in challenge/interview mode.
- [A] Architectures are saved as JSON graphs in D1 and can be revisited, duplicated, or attached to a case-study attempt.
- [B, v1.0] Cost annotation: each node shows an estimated monthly cost from bundled pricing data; the canvas shows total architecture cost in real time.
- [B, v1.0] Version history: every save creates a snapshot; users can compare versions side-by-side and roll back.
- [B, v1.0] Export formats: shareable URL, PNG/SVG image, Mermaid code, Excalidraw file, portfolio PDF.
- [B, v2.0] Collaborative editing via Durable Objects + WebSockets: multiple users edit the same canvas with live cursor presence.

12. Assessment System

v2.1 keeps the v2.0 assessment types but reclassifies the v2.0 additions. Cost reasoning and code pattern recognition are Tier B (require Cost Calculator and Code Walkthrough respectively). Voice explanation is Tier C (requires browser TTS). Tier A MVP ships with the original five assessment types.

Assessment	Measures	Output	Tier
Recall card	Memory of definitions / comparisons	Remembered / Review	A
Concept quiz	Conceptual understanding	Score + weak concepts	A

Assessment	Measures	Output	Tier
Prediction question	Reasoning about behavior	Explanation + result	A
Scenario decision	Choosing a component or pattern	Decision + trade-off quality	A
Architecture review	Systems thinking	Findings + score + rationale	A
Interview (text)	Communication + breadth + depth	Rubric + transcript + follow-ups	A
Cost reasoning	Estimating cost drivers and trade-offs	Estimate + ranking + justification	B
Code pattern recognition	Connecting code to architecture patterns	Pattern ID + alternative implementations	B
Voice explanation	Explaining concepts aloud (browser TTS + STT)	Clarity score + coverage map	C

13. Progress Model

v2.1 returns to the v1.0 five-dimension mastery model for the Tier A MVP: **Learn, Recall, Apply, Explain, Interview**. The Cost and Code dimensions from v2.0 are added in v1.0 alongside the Cost Calculator and Code Walkthrough features. This keeps the Tier A schema simpler and avoids empty-mastery-column problems before the relevant features exist.

14. Gamification

- Streaks with recovery tokens, not harsh reset mechanics. One missed day costs a token, not the streak.
- Concept mastery badges and milestone levels (Apprentice, Builder, Architect, Principal) — tied to demonstrated skill, not time spent.
- XP system tied to demonstrated understanding: completing a quiz earns XP, but only passing with rationale earns bonus XP. Page scrolling earns zero XP.
- Daily completion bonus, but no reward for mindless page scrolling or rapid-fire quiz answering without reasoning.
- Challenge difficulty tiers: Warm-up, Solid, Hard, Interview, Staff (Tier B calibration).
- Achievement categories: Depth (mastered N concepts in one area), Breadth (touched N areas), Application (built N architectures), Consistency (N-day streak).
- Cost and Voice achievement categories added in v1.0 and v2.0 respectively (Tier B/C).
- No public profile, no leaderboard, no sharing pressure. This is a personal app — gamification is for motivation, not social comparison.

15. Non-Functional Product Requirements

Requirement	Target
Core content availability	100% of authored core lessons and diagrams work without third-party content sites. Content is static-bundled at deploy time.
Web performance	LCP target < 2.5s on a good connection for normal content pages. LCP < 1.8s for the Home screen. Static content served from Cloudflare edge.
Interactive lab response	UI state updates in < 100ms for local simulation logic in normal cases. All labs run client-side — zero Worker requests.
Accessibility	Keyboard navigation, visible focus, semantic structure, captions/alt text, reduced motion support, dyslexia-friendly font option, screen reader compatibility.

Requirement	Target
Offline resilience	PWA installable. Service worker caches core shell + current lesson + notes. Full offline is opt-in per lesson pack (Tier B).
Localization	English-only for personal use. i18n-ready architecture (translatable strings separated from structure) in case of future multi-user.
Security	OWASP-aligned practices, least privilege, secure sessions, encrypted transport. No password storage — GitHub OAuth only.
Reliability	Graceful degradation for AI, voice, and enrichment services. No single optional dependency can take down core learning.
Privacy	Notes, annotations, and personal architectures are private by definition (single user). AI history is deletable.
Zero-cost deployment	No paid dependency and no accidental billing. Core functionality remains usable at free-tier exhaustion. Cloudflare's Free plan fails-closed rather than silently switching to paid usage. Optional services have hard quota guards that fail-closed (feature unavailable, not billed) when free quotas are reached.
Quota guard behavior	When a free quota is reached: feature becomes unavailable, friendly fallback is shown, no bill is generated. The learner is never asked to upgrade. Raw quota data lives under Settings → System Health (not prominent in the main UI). Normal state shows a small healthy indicator; near-limit shows a subtle warning; limit-reached shows "AI Mentor is back tomorrow. Core NO CAP features remain fully available."
Free tier realism	The free quotas are comfortably above expected single-user usage — not "effectively unlimited". The limits are real and the quota guards exist to enforce them gracefully.

16. Online Resource Boundary

Permitted optional online features: citation previews (cached via Worker fetch with 5s timeout, 7d TTL in KV), AI services (Workers AI with quota guards), browser-native TTS/STT (no server cost). Core learning content must not depend on those integrations. Every external feature has a timeout, cache, fallback state, and explicit failure UI.

FAILURE UX CONTRACT

When an optional service is unavailable, the UI shows a small, dismissible badge ("Voice unavailable - showing text", "AI quota reached - showing authored explanation") rather than a blocking error. The learner can continue without interruption. This applies to AI, TTS, STT, and external link previews.

17. Version Roadmap

v2.1 rebalances the release plan around the tier classification. Each version is a coherent product on its own — not a partial milestone. The names signal the product identity at each stage.

Version	Name	Scope	Cost
0.1	The Learning Engine	Roadmap, Concepts, Daily Dose, Lessons, Diagrams, Quizzes, Mastery, Review, Glossary, Focus Mode. Personal auth (GitHub OAuth for one user). PWA installable.	Rs 0
0.5	The Design Gym	Interactive Labs (client-side), Architecture Playground (solo + validation), Case Studies, Interview Mode (text), Projects.	Rs 0
1.0	The System Design Workbench	Notes & Annotations, Collections, Cost Calculator (bundled pricing), Career Path Mapping, Code Walkthrough Mode, Export & Share, Real-World Teardowns, version history + portfolio export.	Rs 0
2.0	The Multiplayer / AI Layer	Collaborative Whiteboarding (Durable Objects), Voice / Audio Mode (browser TTS), Voice Interviewer (browser STT), AI Mentor (Workers AI, quota-guarded), Calendar Sync (optional), External Resource Previews (optional).	Rs 0 (with quota guards)
Future	Beyond	Code Sandbox (live execution), Live Cloud Pricing APIs, native mobile app, multi-user / cohorts. Re-evaluate infrastructure cost when constraints change.	TBD

18. Success Metrics

For a personal app, "success metrics" are really "am I actually learning?" checks. v2.1 reframes the v2.0 metrics as personal learning signals rather than business KPIs.

Metric	Why it matters (personal)
7-day learning retention	Did the habit survive the first week?
Concept mastery rate	Am I actually learning, or just scrolling?
Review completion rate	Is spaced repetition working for me?
Architecture challenge completion	Am I moving from theory to application?
Case study completion	Can I do end-to-end system design?
Interview readiness delta	Am I getting better across repeated sessions?
Session return rate	Is Daily Dose creating a durable routine?
Notes & Collections adoption [B]	Am I synthesizing, not just consuming?
Cost reasoning accuracy [B]	Can I estimate real cloud spend within 30%?
Weekly active days	Simple habit signal. Target: 5+ days/week.
Worker request count / day	Infrastructure health. Stay under 100k/day.
D1 rows read / day	Database health. Stay under 5M/day.

19. Risks and Mitigations

Content explosion → Define a strict schema and authoring templates; ship core concepts first. Use the typed content schema so new concepts do not require new UI code.

Too much gamification → Reward demonstrated skill, not screen time. XP only for completed reasoning tasks, never for passive actions.

AI dependency → Keep authored content and deterministic evaluation as the foundation. AI never mutates mastery scores directly. Quota guards fail-closed.

Interactive labs become shallow toys → Tie each lab directly to a concept and a measurable phenomenon. Every lab has a success criterion and a reflection prompt.

Roadmap becomes overwhelming → Show next best action rather than entire course pressure. Default to Guided Path view; Explore Graph is opt-in.

Scope creep → Hold the product identity lock: NO CAP is the system-design gym. Tier D features stay deferred regardless of cool-factor.

Free tier exhaustion → Quota guards on every optional service. Dashboard shows daily Worker/DI/AI usage. Hard fail-closed, never bill.

Browser TTS/STT quality varies → Tier C feature — if browser support is poor, the feature simply is not offered. Never block core learning on voice.

Bundled pricing data goes stale → Versioned dataset updated quarterly via repo PR. "As of" date shown on every cost estimate. Major changes trigger revalidation prompt.

Collaboration infra is complex → Build on Durable Objects (Cloudflare-managed). CRDT library (Yjs) is battle-tested. Cap concurrent editors at 4 for personal use.

Glassmorphism hurts readability → Glass effects are limited to surfaces over imagery; over solid backgrounds we use solid cards. WCAG AA contrast is enforced for all text.

Python Workers limitations → Pyodide runtime does not support all CPython packages. Keep FastAPI domain logic pure-Python; avoid native extensions. Fallback to TypeScript Worker if needed.

20. Definition of Done for a Learning Concept

A concept is "done" (ready to publish) when all of the following exist:

- A concise explanation (1-paragraph summary, scannable in 30 seconds). [A]
- A deep explanation (multi-section lesson with subsections for problem, mechanism, trade-offs, failure modes). [A]
- At least one visual or flow exists (diagram, sequence, state machine, or interactive widget). [A]

- Trade-offs and failure modes are documented explicitly — never implied. [A]
- At least one interactive or prediction activity exists for major concepts. [A]
- At least five assessment items exist with explanations and rationale. [A]
- At least two real-system mappings exist (where this concept appears in production). [A]
- Prerequisites and related concepts are linked bidirectionally. [A]
- Interview prompts and common misconceptions are included. [A]
- Review scheduling metadata is present (initial interval, ease, decay rules). [A]
- A code walkthrough snippet exists for implementation-heavy concepts (read + modify + predict, no execution). [B]
- A cost reasoning prompt exists for infra-heavy concepts (Cache, DB, Queue, CDN, LB). [B]
- A voice-renderable version of the lesson is available (browser TTS-compatible prose, alt-text for diagrams). [C]

21. Product Recommendation

BUILD ORDER

Build 0.1 (Learning Engine) completely before touching 0.5. Build the content model, roadmap, concept graph, Daily Dose, assessment engine, and mastery model before spending any effort on visual polish or simulations. Those systems make later features much easier to add without rewriting the product. The Cloudflare free tier is generous for a single user — you will not hit limits in 0.1. You might in 2.0 (AI + collab), which is why those features are quota-guarded from day one.